Informe de arquitectura: PixelArt colaborativo global

Oct 14, 2026 · @Claudia

1. Introducción

La arquitectura propuesta es **multi-región activo-activo**: varias regiones del mundo atienden usuarios al mismo tiempo detrás de Cloudflare, con backend en **Go**, lienzo en memoria con **Redis** e historial en **ScyllaDB**.

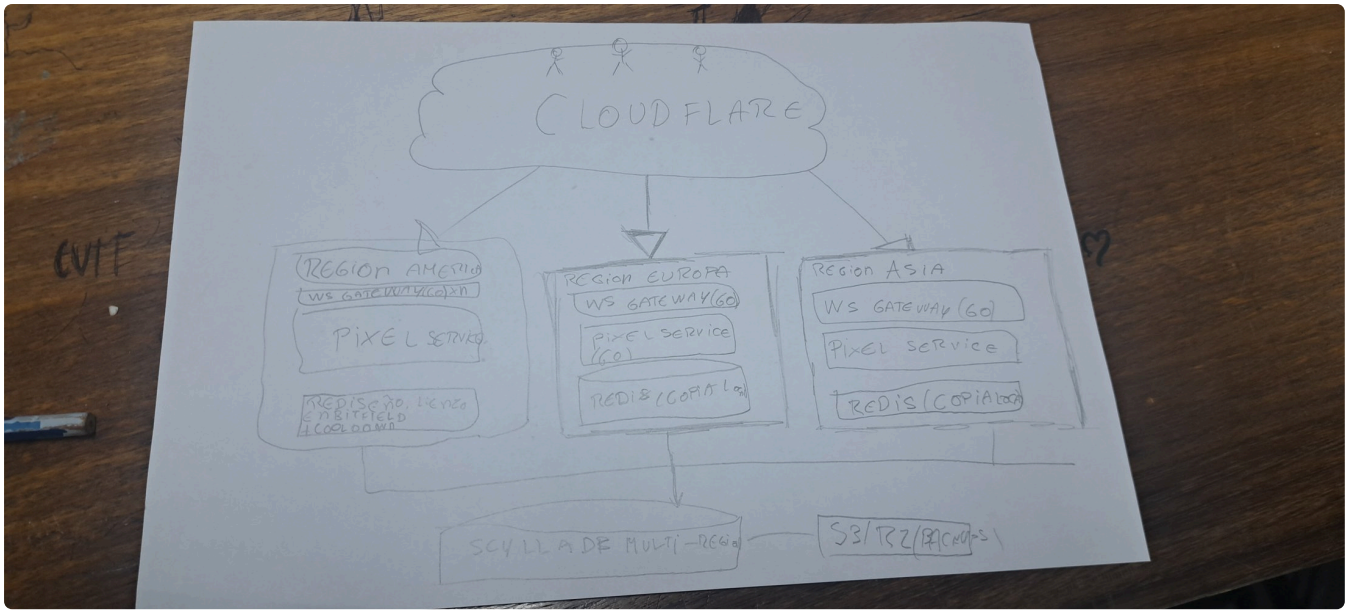
El proyecto es una aplicación web donde miles de personas pintan pixeles de forma simultánea sobre un lienzo compartido, para crear un pixel art en comunidad (estilo *r/place*). Cada usuario pinta un pixel cada 5 minutos y ve en tiempo real lo que pintan los demás.

Requisito no funcional	Meta concreta
Alta disponibilidad	99,9% anual: como máximo 8 h 45 min de caída al año (43 min al mes)
Baja latencia	Pixel confirmado en menos de 100 ms y visible en todo el mundo en menos de 300 ms
Réplicas a nivel mundial	El lienzo completo existe en cada región
Copias de seguridad	Pérdida máxima de 5 minutos de lienzo (RPO)
Resiliencia	La caída de un servidor, una zona o una región completa no detiene el servicio

Este informe explica el boceto hecho a mano, el funcionamiento del sistema y el fundamento de cada tecnología elegida.

2. Boceto hecho a mano

El boceto muestra la arquitectura en cuatro niveles: usuarios, Cloudflare, tres regiones activas y la capa de datos con sus respaldos.



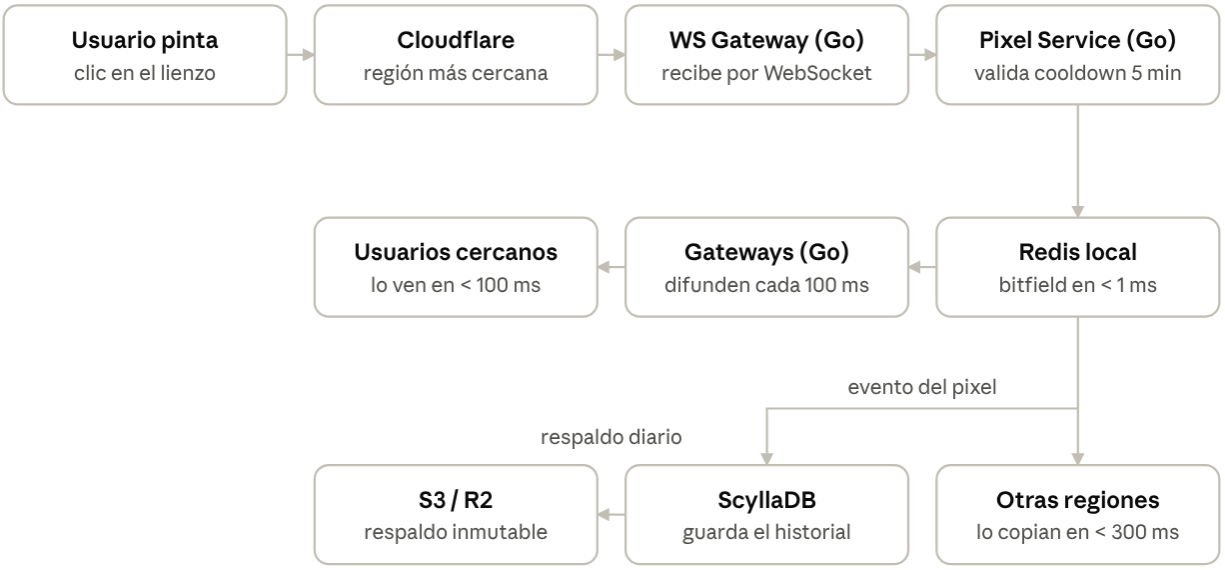
Elemento dibujado	Qué representa
Monitos sobre la nube	Los miles de usuarios que pintan desde el navegador al mismo tiempo.
Nube "Cloudflare"	La puerta de entrada mundial: DNS, CDN, protección contra ataques y envío de cada usuario a la región más cercana.
Tres flechas desde la nube	El tráfico se reparte entre las tres regiones; todas reciben usuarios a la vez.
Región América	WS Gateway (Go) × N, Pixel Service y Redis en São Paulo, con el lienzo en bitfield y el cooldown.
Región Europa	El mismo stack: WS Gateway (Go), Pixel Service (Go) y una copia local de Redis.
Región Asia	El mismo stack: WS Gateway (Go), Pixel Service y una copia local de Redis.
Línea horizontal bajo las regiones	La conexión que une las tres regiones y las lleva a la base de datos compartida.
Cilindro "ScyllaDB multi-región"	La base de datos durable con el historial de pixeles y los usuarios, replicada en cada región.
Caja "S3/R2 (backups)"	El almacenamiento de las copias de seguridad, conectado a ScyllaDB.

Que las tres regiones tengan exactamente los mismos componentes es intencional: cualquiera puede reemplazar a otra si falla.

3. Funcionamiento: el recorrido de un pixel

Un pixel pintado se confirma en su región en menos de 100 ms y llega a todas las demás en menos de 300 ms, siguiendo las flechas del boceto de arriba hacia abajo.

Un pixel se ve en su región en menos de 100 ms y en el mundo en menos de 300 ms



Recorrido de un pixel · de un clic a todo el mundo

Si el usuario todavía está en su tiempo de espera, Pixel Service rechaza el pixel antes de escribir nada. Guardar en ScyllaDB ocurre en segundo plano, así que el usuario nunca espera a la base de datos.

4. Decisiones tecnológicas y fundamentos

Cada tecnología se eligió a partir de una característica del problema: el lienzo completo pesa solo 4 MB, se escribe poco y se lee muchísimo, y si dos personas pintan el mismo pixel simplemente gana la última.

Esa última característica es clave. Según el **teorema CAP**, no se puede tener a la vez consistencia fuerte y disponibilidad total ante cortes de red. Como un pixel no necesita consistencia fuerte entre continentes, el diseño prioriza **disponibilidad y tolerancia a particiones (AP)**, que es lo que piden los requisitos.

Componente	Elección	Fundamento	Alternativas descartadas
Borde global	**Cloudflare**	Red Anycast con más de 300 puntos de presencia: el usuario se conecta al más cercano. Hace el failover entre regiones con health checks, sirve el lienzo como imagen desde la CDN y frena ataques DDoS y bots.	DNS de un solo proveedor de nube: no trae CDN ni anti-DDoS integrados.
Tiempo real	**WebSocket**	Una conexión abierta y bidireccional; los pixeles llegan al instante en mensajes de pocos bytes.	Polling: 100.000 usuarios preguntando cada segundo generan 100.000 peticiones por segundo casi siempre vacías.
Backend	**Go** sobre Kubernetes	Cada conexión es una goroutine de unos 4 KB, así que un servidor aguanta ~50.000 usuarios. Pausas de memoria menores a 1 ms y arranque en menos de 1 s para autoescalar.	Node.js (un hilo por proceso), Elixir (menos desarrolladores disponibles), serverless (no sirve para conexiones largas).
Datos en vivo	**Redis / Valkey**	El lienzo de 2000 × 2000 pixeles cabe en 4 MB de memoria. Pintar un pixel con BITFIELD tarda menos de 1 ms, y el cooldown de 5 minutos se controla con una clave que expira sola.	Leer el lienzo desde una base de datos en disco: cientos de veces más lento.
Datos durables	**ScyllaDB** multi-región	No tiene servidor maestro: cualquier región escribe, así que si una cae no hay que cambiar nada. Replica entre continentes y resuelve conflictos con "gana el último".	PostgreSQL y MongoDB (un solo primario que puede caer), CockroachDB y Spanner (cada escritura espera a otros continentes).
Respaldos	**S3 / R2**	Almacenamiento barato y muy durable, con versionado, copias inmutables y replicación a otra región.	Respaldos en el mismo servidor: se pierden junto con él.

El backend se divide en dos servicios separados, como en el boceto: **WS Gateway** mantiene las conexiones y crece con la cantidad de usuarios, mientras que **Pixel Service** valida y guarda los pixeles y crece con la cantidad de escrituras. Ambos son sin estado, así que cualquier copia puede caer y otra la reemplaza.

5. Cómo el diseño cumple cada requisito

Los cinco requisitos se cumplen con elementos que ya aparecen en el boceto.

Requisito	Cómo se cumple
Disponibilidad 99,9%	Tres regiones activas a la vez. Si cada una tuviera solo 99,5% de disponibilidad, la probabilidad de que caigan las tres juntas es $0{,}005^3$, unas 1 en 8 millones. Cloudflare detecta la región caída y envía a sus usuarios a otra en menos de 30 segundos.
Baja latencia	Cada usuario se conecta a la región más cercana. El lienzo está en memoria (Redis, menos de 1 ms) y la conexión WebSocket queda abierta, sin reconectar en cada pixel.
Réplicas mundiales	Cada región tiene su copia completa del lienzo en Redis, y ScyllaDB replica el historial en todas las regiones.
Copias de seguridad	Foto del lienzo cada 5 minutos en S3/R2, respaldo diario de ScyllaDB y respaldo incremental cada hora. Las copias son inmutables (nadie puede borrarlas, ni un ransomware) y se replican a otra región: regla 3-2-1.
Resiliencia	Los servicios en Go no guardan estado, así que Kubernetes reemplaza cualquier copia caída en segundos. ScyllaDB no tiene maestro, así que no hay un punto único de falla en los datos.

Ante fallas, el sistema se degrada en vez de caerse:

Si falla…	El usuario…
Un servidor	Se reconecta a otro en ~1 segundo.

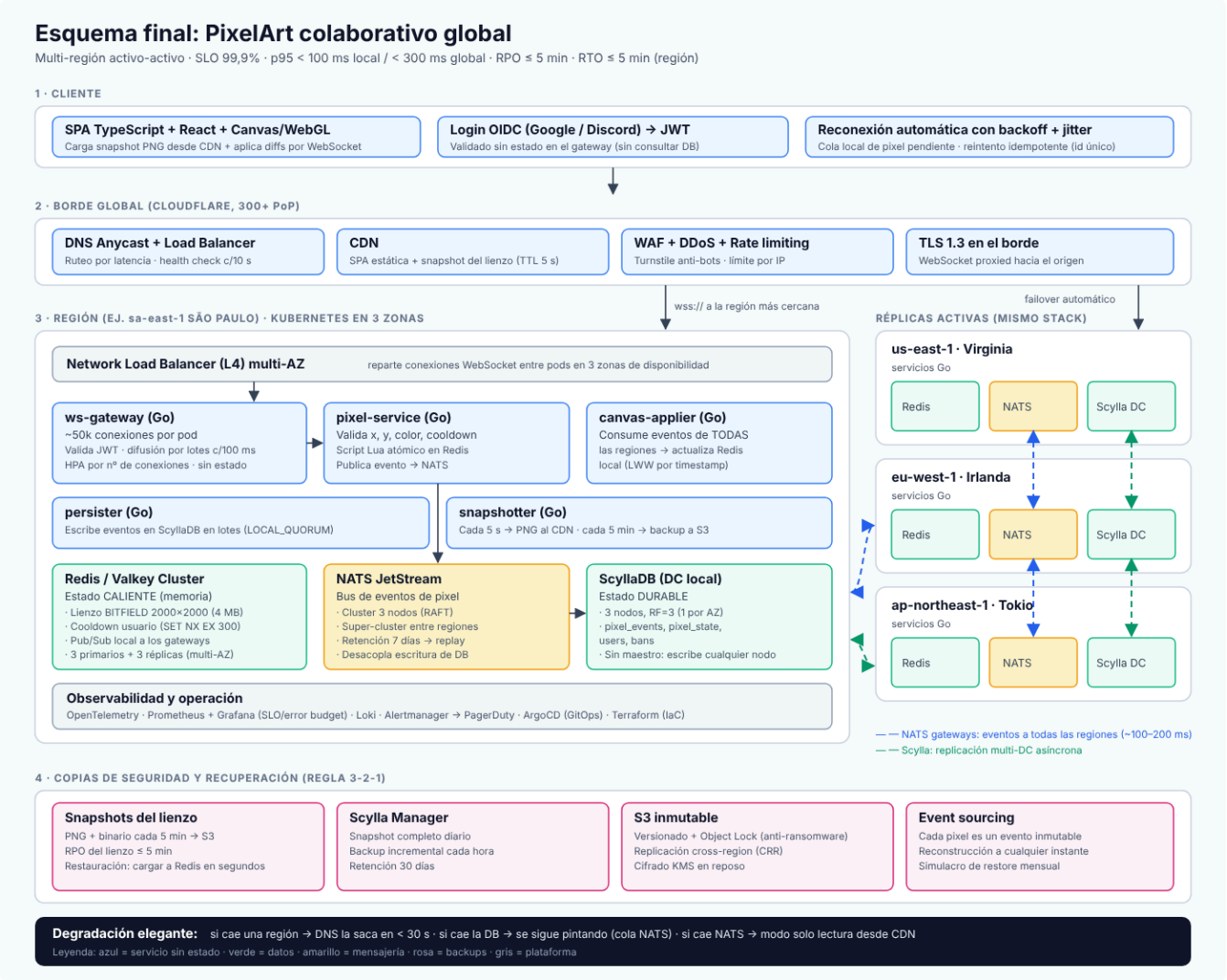
Si falla…	El usuario…
Una región completa	Es enviado a la siguiente región más cercana; nota algo más de latencia.
La base de datos	Sigue pintando: el lienzo vive en Redis y los cambios se guardan al volver.
Todo el backend	Sigue viendo el lienzo, porque la imagen se sirve desde la CDN de Cloudflare.

Una réplica no es un respaldo: si alguien borra datos, la réplica también los borra. Por eso el diseño tiene ambos, y la restauración se prueba una vez al mes.

6. Esquema final: mejoras respecto al boceto

El esquema final mantiene la estructura del boceto y agrega cinco piezas que lo hacen apto para producción.

Mejora	Qué resuelve
NATS JetStream entre regiones	En el boceto la línea que une las regiones no dice cómo viajan los pixeles. NATS publica cada pixel como evento y lo entrega a todas las regiones en ~150 ms; si la base de datos falla, los eventos esperan en cola sin perderse.
Cuarta región (Estados Unidos)	Divide América en São Paulo y Virginia, para bajar la latencia en Norteamérica.
Kubernetes en 3 zonas por región	Cada región sobrevive a la caída de un centro de datos completo sin tener que pasar a otra región.
Snapshotter	Genera la imagen del lienzo para la CDN cada 5 segundos y la copia de seguridad cada 5 minutos.
Observabilidad (Prometheus, Grafana, OpenTelemetry)	Mide la disponibilidad real contra la meta del 99,9% y alerta antes de que el usuario note un problema.



7. Conclusión

La arquitectura cumple los cinco requisitos porque no depende de ningún componente único: cada región es una copia completa del sistema y la base de datos no tiene maestro.

La decisión central fue separar el estado en dos niveles. El lienzo vive en memoria (Redis), lo que da la baja latencia, y el historial vive en una base de datos replicada (ScyllaDB), lo que da la durabilidad. Como un pixel admite que "gane el último", el sistema puede priorizar disponibilidad sin perder corrección.

Respuesta directa a la pregunta del trabajo: para el backend, **Go** con WebSocket sobre Kubernetes; para la base de datos, **Redis/Valkey** para el lienzo en vivo y **ScyllaDB** multi-región para el historial, con respaldos inmutables en **S3/R2**.